

8.3 LoRA, Adapters, and Federated Updates

Train BA , freeze W . $r(d+k)$ trainable, not dk . Merge has no extra decode latency.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

Full fine-tuning moves every entry of W . **LoRA** (Hu et al., 2021) freezes W and learns a **low-rank residual**. You store a few megabytes per task instead of a second copy of the 7B.

First time this method appears. LoRA freezes W and trains a low-rank residual BA .

What. $\Delta W \approx BA$ with rank $r \ll \min(d, k)$. Store megabytes per task, not a second 7B. **Why.** Full fine-tune copies every weight (GPT-3: 175B extra per task) and overwrites (note 8.2). **Architecture.** Insert A, B on attention and/or MLP projections. Init $B = 0$ so you start as the base. QLoRA: 4-bit W , float adapters. **How.** Train only A, B . Merge $W \leftarrow W + (\alpha / r)BA$ at deploy (no extra latency) or swap adapters. **Formula.** $h = Wx + (\alpha / r)BAx$. Trainable count per matrix $r(d + k)$, not dk . **Tradeoffs.** + Cheap, mergeable, one adapter per skill. – Small r may fail hard domain shifts; adapters can leak in federated settings; not a new architecture.

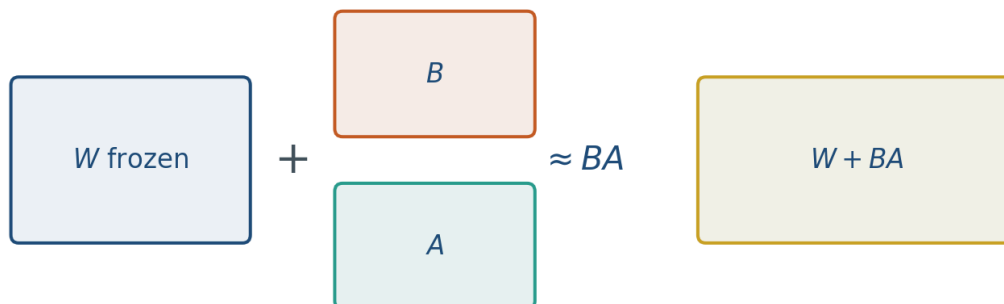
1. $W + BA$

For a linear map $W \in \mathbb{R}^{d \times k}$,

$$h = Wx + \frac{\alpha}{r}BAx,$$

with $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$, rank $r \ll \min(d, k)$. Initialize $B = 0$ so the adapter starts as the base model. Train only A, B (and maybe a bias). Attention projections and MLP up/down maps are the usual insertion points.

LoRA: train a thin pair BA , leave W alone



CS224N L11: a full copy of W is 175B extra for GPT-3. LoRA is this table.

$d = k = 4096, r = 8$	count
dk	$4096^2 = 16.8\text{M}$
$r(d + k)$	$8 \cdot 8192 = 65,536$
ratio	0.39%

QLoRA quantizes W to 4-bit and still trains float adapters. Same algebra. Lab 8 uses a 4×4 frozen W so you can print BA .

Trainable count for one matrix is $r(d + k)$, not dk . That is the whole point.

2. Rank, merge, many tasks

The reason for LoRA is that a full Δ has the same size as W (GPT-3: 175B extra weights per task).

Encode $\Delta W = BA$ with $r \ll \min(d, k)$. Merge at deploy and there is **no extra inference latency**; swap adapters by subtracting one BA and adding another.

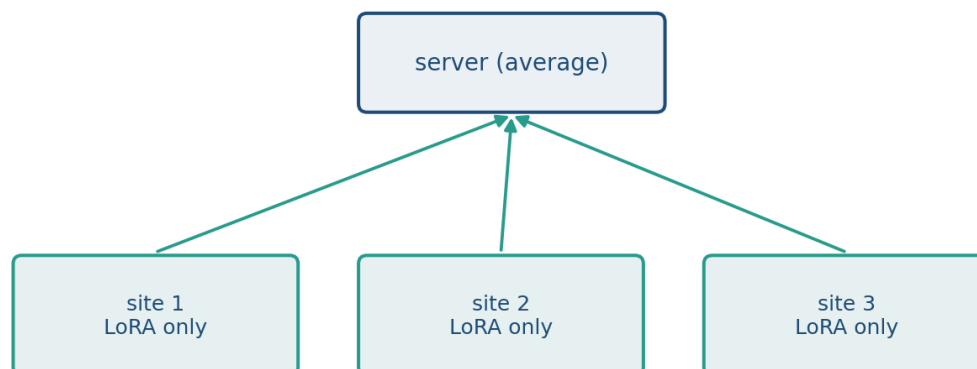
Small r (4, 8, 16) is often enough for style and instruction shifts; harder domain shifts may want larger r or more layers. At deploy time you can **merge** $W \leftarrow W + (\alpha / r)BA$ and throw the adapter away, or keep several adapters and swap them.

One adapter per skill is how you avoid the XOR-then-AND overwrite (note **8.2**): the base stays, the skill is a file.

3. Federated adapters

Sites that cannot ship raw text can still train a LoRA locally and send A, B to a server. The server averages adapters (or stacks them) without seeing documents. That is **federated** fine-tuning in the sense this course needs: communication is the adapter, not the corpus. Privacy is not automatic (adapters can leak), but the footprint is the right size.

Federated sketch: ship adapters, not raw documents



For a project: report r , which modules you adapted, α , and whether you merged. Compare to a small full fine-tune if you can afford it.

4. Teaching this note

About **35–40 minutes** at the board: write $r(d + k)$ vs dk , init $B = 0$, merge vs swap. Play the **full** LoRA video as assigned (in class, first 20–25 min if the clip runs long; rest after). Lab 8 section 1 is the 4×4 printout.

5. Worked example

One map, $d = k = 4096$, rank $r = 8$:

$$r(d + k) = 8 \cdot (4096 + 4096) = 65,536$$

trainable numbers. Full W has $4096^2 = 16,777,216$ entries. Ratio $65536 / 16777216 \approx 0.0039$ (about **0.39%**). α / r is a scale; it does not add parameters.

Lab size: $r = 1$, $d = k = 4$. $A \in \mathbb{R}^{1 \times 4}$ (4 numbers), $B \in \mathbb{R}^{4 \times 1}$ (4 numbers), **8** trainable vs **16** in W . At init $B = 0 \Rightarrow BA = 0 \Rightarrow h = Wx$.

Why not init $A = 0$ instead? Either factor zero works for a zero start; the usual recipe is Gaussian A and **zero** B so the first step can still move (nonzero A , B getting a gradient). If both start at 0, the product stays 0 until you break the deadlock (you would not).

Merge: $W' = W + (\alpha / r)BA$ is $d \times k$ again. Inference cost matches the base; you lose easy unmerge unless you kept A, B .

6. Where students get stuck

- Counting LoRA as r^2 or as $2r$ only. It is $r(d + k)$ per adapted matrix.

- Init both factors randomly so the model is not the base at step 0.
 - Shipping a hospital adapter and calling it private. The corpus stayed, but adapters can still leak.
-

7. Video

[Umar Jamil — LoRA, explained visually + PyTorch from scratch](#). Play the **full** video as assigned. In a two-hour class, run **0:00–~25:00** (math + the BA diagram) and leave the PyTorch walkthrough for after, or play through if time. Pause when B is zeros: that is Lab 8's first print.

8. Practice

1. If $r = 1$ and $d = k = 4$, how many trainable numbers are in (A, B) versus W ?
2. Why initialize $B = 0$ rather than $A = 0$?
3. A hospital trains a LoRA and sends it to you. What did they *not* send, and what can still leak?
4. $d = 1024$, $k = 4096$, $r = 16$. Compute $r(d + k)$ and the ratio to dk .
5. You adapt q, k, v, o projections, each $d \times d$ with $d = 2048$, $r = 8$. Ignore α . How many trainable parameters is that (four matrices)?